

Reliability Analytics Dashboard

Complete Hackathon Workflow Blueprint

Goal: Give local vendors data to improve, not just complaints to ignore.

Area	Decision
Frontend	React + Vite + Tailwind CSS + Recharts + Axios
Backend	Node.js + Express + CORS
Data	Mock JSON only, with 10 vendor entries
Frontend Port	http://localhost:5173
Backend Port	http://localhost:5000
Deployment	Vercel for frontend, Render for backend

1. Problem Understanding

Local vendors in hyperlocal marketplaces have very little visibility into delivery success, complaint patterns, response time, ratings, and overall reliability. The solution is a dashboard that converts raw vendor performance data into simple, visual, actionable insights.

- Admins can monitor all vendors and identify low performers.
- Vendors can understand their own reliability and weak areas.
- Judges see a working, visual, demo-friendly analytics product.

2. User Roles And RBAC

Feature	Admin	Vendor
View all vendors	Allowed	Not allowed
View own vendor profile	Allowed	Allowed
View platform summary	Allowed	Not allowed
Search and sort vendors	Allowed	Not allowed
View complaint comparison chart	Allowed	Own data only
View reliability score	All vendors	Own score only
Flag low performers	Allowed	Not allowed
Export reports	Future feature	Future feature

3. User Journey And Platform Workflow

Admin journey:

- Open dashboard and view summary cards.
- Compare vendors using charts and table.
- Search or sort vendors by score, complaints, or rating.
- Open vendor detail page for deeper analysis.
- Identify Good, Average, and Poor performers.

Vendor journey:

- Open own vendor detail page.
- Check reliability score and status badge.
- Review completed vs failed orders.
- Analyze weekly delivery rate, complaints, response time, and rating.
- Use the dashboard to understand improvement areas.

```
vendors.json
-> Express API computes score/status
-> React fetches data using Axios
-> Dashboard renders cards, charts, table, and detail views
```

4. File Structure

```
reliability-dashboard/
├── data/
│   ├── vendors.json      ← includes weeklyData array per vendor
│   └── complaints.json
├── server/
│   ├── index.js
│   ├── routes/
│   │   ├── vendors.js
│   │   └── stats.js
│   └── src/
│       ├── components/
│       │   ├── Navbar.jsx
│       │   ├── SummaryCard.jsx
│       │   ├── VendorTable.jsx
│       │   ├── StatusBadge.jsx
│       │   └── ChartCard.jsx
│       ├── pages/
│       │   ├── Dashboard.jsx
│       │   └── VendorDetail.jsx
│       ├── App.jsx
│       └── main.jsx
└── package.json
```

5. Data Model

Each vendor object must include delivery rate, average response time, rating, and embedded weeklyData. The project uses 10 vendors for the MVP.

```
{
  "id": "v001",
  "name": "SupplyCo",
  "reliabilityScore": 87,
  "status": "good",
  "totalOrders": 320,
  "completedOrders": 278,
  "failedOrders": 42,
  "deliveryRate": 86.9,
  "totalComplaints": 5,
  "avgResponseTime": 12,
  "rating": 4.2,
  "region": "North",
  "weeklyData": [
    { "day": "Mon", "deliveryRate": 85, "complaints": 1 },
    { "day": "Tue", "deliveryRate": 90, "complaints": 0 }
  ]
}
```

6. API Contract And Backend Routes

Endpoint	Purpose	Used By
GET /api/vendors	All vendors with computed reliabilityScore and status	Dashboard, table, charts
GET /api/vendors/:id	Single vendor details with weeklyData	Vendor detail page
GET /api/summary	totalVendors, avgReliabilityScore, topPerformer, totalComplaints	Summary cards
GET /api/stats/complaints	Complaint analytics	Complaints chart
GET /api/stats/deliveries	Delivery trend analytics	Delivery line chart

Route file plan:

```

routes/vendors.js
■■■ GET /api/vendors
■■■ GET /api/vendors/:id

routes/stats.js
■■■ GET /summary      → mounted as /api/summary
■■■ GET /complaints   → mounted as /api/stats/complaints
■■■ GET /deliveries    → mounted as /api/stats/deliveries

```

Reliability calculation:

```

reliabilityScore = Math.round((completedOrders / (completedOrders + failedOrders)) * 100)

score > 80 = Good
score 60-80 = Average
score < 60 = Poor

```

7. Frontend Component Plan

Page/Component	Responsibilities
Dashboard.jsx	Fetch summary/vendors, render cards, charts, search, sort, table
VendorDetail.jsx	Fetch single vendor, render score badge, pie chart, line chart, detail cards
SummaryCard.jsx	Reusable metric card for dashboard and vendor detail
VendorTable.jsx	Vendor rows with score, rating, complaints, and status badge
StatusBadge.jsx	Color-coded Good/Average/Poor display
ChartCard.jsx	Consistent chart container

Vendor Detail must include:

- DeliveryLineChart using vendor weeklyData.
- SummaryCard - Avg Response Time using avgResponseTime.
- SummaryCard - Rating using rating.
- SummaryCard - Total Orders using totalOrders.
- PieChart for completedOrders vs failedOrders.

8. Dashboard And Analytics Requirements

- Four summary cards: Total Vendors, Avg Reliability, Top Performer, Total Complaints.
- Bar chart for complaints per vendor.
- Line chart for weekly delivery rate trend.
- Vendor table with name, score, rating, complaints, and status.
- Search filter by vendor name.
- Sort by reliability score, complaints, or rating.
- Responsive design for laptop and mobile demo.

9. Security, Authentication, And Verification

- MVP can simulate roles using a simple Admin/Vendor selector.
- Backend validates vendor ID and returns 404 for invalid vendors.

- CORS allows frontend to call backend.
- Use environment variable for frontend API base URL when deploying.
- Future version can add JWT login, bcrypt password hashing, OTP verification, protected routes, and audit logs.

10. Team Split

Team Member	Ownership	Tasks
Person 1	/data, README, Git, Deployment	Create 10 vendors, merge branches, deploy, prepare screenshots
Person 2	/src	Build React pages, components, charts, search/sort, UI polish
Person 3	/server	Build Express routes, scoring logic, status logic, summary API

11. Demo Checklist

- GET /api/vendors returns 10 vendors.
- GET /api/summary returns correct summary data.
- GET /api/vendors/:id returns weeklyData.
- Vendor table shows 10 rows.
- Search and sort work.
- Charts render with real data.
- Vendor detail page shows avg response time, rating, and total orders.
- Frontend and backend are deployed or localhost backup is ready.

12. Future Scalability Ideas

- Add real database using MongoDB or PostgreSQL.
- Add vendor/admin login with JWT and RBAC.
- Add ML prediction for future reliability score.
- Add automated alerts for poor performers.
- Add complaint category analysis and sentiment analysis.
- Add exportable PDF/CSV reports.
- Add location-wise and delivery-agent-wise analytics.

Final MVP Goal

A working Reliability Analytics Dashboard where admins can view vendor performance, compare reliability metrics, identify weak vendors, and open vendor detail pages with clear charts and scores.